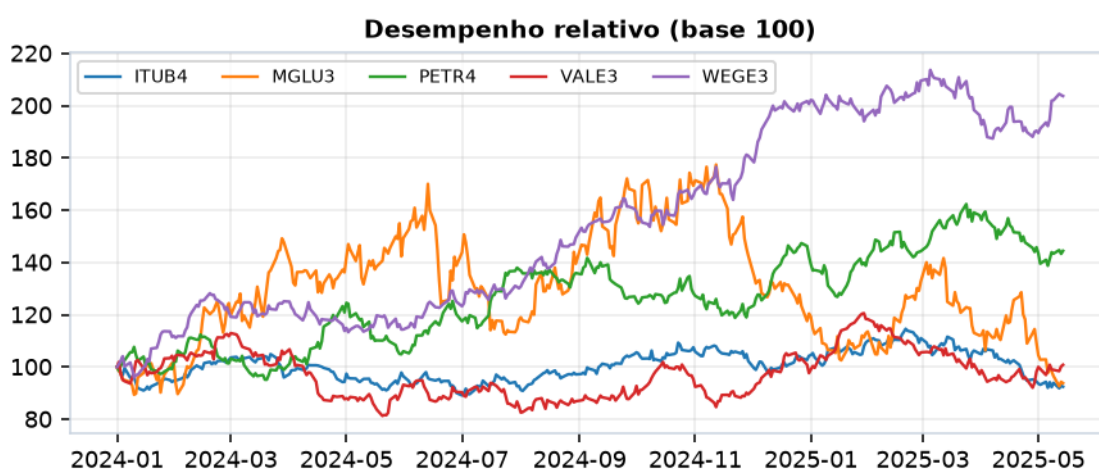


APOSTILA SQL + PYTHON + SQLITE

Gráficos financeiros na prática — dezenas de exemplos
comentados linha por linha, em português



Um dos 16 gráficos construídos ao longo desta apostila.

Conteúdo: modelagem de um banco de cotações, SQL de finanças (window functions, CTEs, agregações), ponte SQLite → pandas → matplotlib e 16 gráficos completos: linha, candlestick, volume, médias móveis, retorno mensal, pizza de carteira, lucro/prejuízo, despesas empilhadas, correlação, drawdown, histograma, dividendos, juros compostos, fluxo de caixa, boxplot e sazonalidade.

Sumário

1. Por que SQLite + Python para finanças
2. Preparando o ambiente
3. Modelando o banco financeiro (DDL)
4. Populando o banco com dados
5. SQL essencial para finanças
6. A ponte SQL → pandas → matplotlib
7. Gráfico de linha: histórico de preços
8. Comparando ativos (base 100)
9. Candlestick + volume
10. Médias móveis com window functions
11. Retorno mensal em barras
12. Pizza: composição da carteira
13. Lucro/prejuízo por ativo
14. Despesas mensais empilhadas
15. Matriz de correlação (heatmap)
16. Drawdown
17. Histograma de retornos
18. Dividendos por ação
19. Juros compostos (simulação)
20. Fluxo de caixa: barras + linha acumulada
21. Boxplot de volatilidade
22. Sazonalidade por dia da semana
23. Leitura em lotes (chunks) de bases grandes
24. Boas práticas, desempenho e exportação

1. Por que SQLite + Python para finanças

O trio SQL, pandas e matplotlib resolve 95% das análises do dia a dia.

O **SQLite** é um banco de dados relacional completo que vive dentro de um único arquivo .db. Não há servidor para instalar, usuário para configurar nem porta para liberar: você abre o arquivo e já está conectado. Para dados financeiros isso é perfeito, porque as séries de cotações são naturalmente tabulares (ticker, data, preço, volume) e cabem confortavelmente em milhões de linhas dentro de um arquivo local.

O papel de cada peça é bem definido e vale memorizar:

- **SQL (SQLite)** — filtra, agrupa, junta tabelas e calcula agregações. Faz o trabalho pesado sobre muitos registros.
- **pandas** — recebe o resultado da consulta já reduzido e faz os ajustes finais (índices de data, percentuais, pivots).
- **matplotlib** — desenha. Recebe números prontos e devolve o gráfico.

Dica de ouro

Regra prática: **agregue no SQL, formate no pandas, desenhe no matplotlib**. Trazer 5 milhões de linhas para o Python só para somar depois é o erro mais comum de quem começa — e o mais caro em memória.

Ao longo da apostila usaremos sempre o mesmo banco de exemplo, com 5 ativos da bolsa brasileira e cerca de 500 pregões, além de tabelas de carteira, despesas e dividendos. Todo código é copiável e roda sem internet.

2. Preparando o ambiente

O módulo `sqlite3` já vem com o Python. Só precisamos instalar as bibliotecas de dados e gráficos:

PYTHON

```
# Instale as dependências (uma única vez), no terminal:
# pip install pandas matplotlib
#
# sqlite3 NÃO precisa ser instalado: faz parte da biblioteca padrão do Python.

import sqlite3                # driver do banco embutido no Python
import pandas as pd           # tabelas em memória (DataFrame) e leitura de SQL
import matplotlib             # motor gráfico
matplotlib.use("Agg")         # backend sem janela: salva em arquivo (ideal em servidor)
import matplotlib.pyplot as plt # a API de desenho que usaremos em toda a apostila

print(sqlite3.sqlite_version) # confere a versão do engine SQLite disponível
```

2.1 Abrindo e fechando a conexão com segurança

PYTHON

```
# Forma recomendada: 'with' garante commit/rollback e fecha o recurso mesmo se der erro.
with sqlite3.connect("financas.db") as con: # cria o arquivo se ele não existir
    con.execute("PRAGMA journal_mode = WAL;") # melhora leitura simultânea com escrita
    con.execute("PRAGMA foreign_keys = ON;") # ativa a checagem de chaves estrangeiras
    df = pd.read_sql_query("SELECT 1 AS ok;", con) # consulta de teste vira DataFrame
print(df)                                         # ok
                                                # 0  1
```

Nota

`pd.read_sql_query` é o atalho que usaremos em **todos** os exemplos: ele executa o SQL e já devolve um DataFrame pronto para o matplotlib.

3. Modelando o banco financeiro (DDL)

Um bom gráfico começa numa boa tabela. Este é o esquema usado na apostila inteira — cinco tabelas simples que cobrem preços, posição, proventos e orçamento pessoal.

SQL

```
-- Tabela mestre de ativos: uma linha por papel negociado.
CREATE TABLE IF NOT EXISTS ativos (
    ticker TEXT PRIMARY KEY,      -- código de negociação, ex.: 'PETR4' (chave primária)
    nome TEXT NOT NULL,           -- razão social resumida, usada nas legendas dos gráficos
    setor TEXT                     -- setor econômico: permite agrupar a carteira por segmento
);

-- Série histórica de preços (OHLCV): a tabela que mais cresce.
CREATE TABLE IF NOT EXISTS cotacoes (
    ticker TEXT NOT NULL,         -- a qual ativo a linha pertence
    data TEXT NOT NULL,          -- data no formato ISO 'AAAA-MM-DD' (ordena como texto!)
    abertura REAL,               -- preço do primeiro negócio do dia
    maxima REAL,                 -- maior preço do dia (topo do pavio do candle)
    minima REAL,                 -- menor preço do dia (fundo do pavio)
    fechamento REAL NOT NULL,    -- preço de fechamento: base de quase toda análise
    volume INTEGER,              -- quantidade financeira/negociada no dia
    PRIMARY KEY (ticker, data),   -- impede duplicar o mesmo pregão do mesmo papel
    FOREIGN KEY (ticker) REFERENCES ativos(ticker) -- integridade com a tabela mestre
);

-- Índice que acelera o filtro mais comum: "um ativo, ordenado por data".
CREATE INDEX IF NOT EXISTS idx_cot_ticker_data ON cotacoes(ticker, data);

-- Posição atual do investidor.
CREATE TABLE IF NOT EXISTS carteira (
    ticker TEXT PRIMARY KEY,      -- um registro consolidado por ativo
    quantidade INTEGER NOT NULL,  -- número de ações em custódia
    preco_medio REAL NOT NULL     -- custo médio de aquisição (para calcular lucro/prejuízo)
);

-- Proventos pagos por ação.
CREATE TABLE IF NOT EXISTS dividendos (
    ticker TEXT NOT NULL,         -- ativo pagador
    data TEXT NOT NULL,          -- data do pagamento (ISO)
    valor REAL NOT NULL          -- valor em R$ POR AÇÃO
);

-- Orçamento doméstico, usado nos gráficos de despesas e fluxo de caixa.
CREATE TABLE IF NOT EXISTS despesas (
    data TEXT NOT NULL,          -- data do gasto (ISO)
    categoria TEXT NOT NULL,     -- 'Moradia', 'Alimentação', 'Transporte'...
    valor REAL NOT NULL          -- valor em reais
);
```

Atenção

Guarde datas **sempre** como texto ISO '2025-03-09'. O SQLite não tem tipo DATE nativo, mas esse formato ordena corretamente como texto e funciona com `strftime()` e `substr()`. Formatos como '09/03/2025' quebram a ordenação.

3.1 Executando o DDL a partir do Python

PYTHON

```
con = sqlite3.connect("financas.db") # abre (ou cria) o arquivo do banco
con.executescript(DDL)               # executescript roda VÁRIOS comandos separados por ';'
con.commit()                         # confirma a transação (sem isso, nada é gravado)
```

4. Populando o banco com dados

4.1 Inserindo linha a linha e em lote

SQL

```
-- Inserção simples, uma linha:
INSERT INTO ativos (ticker, nome, setor)
VALUES ('PETR4', 'Petrobras PN', 'Energia');

-- Inserção de várias linhas de uma vez (menos idas ao disco):
INSERT INTO ativos (ticker, nome, setor) VALUES
('VALE3', 'Vale ON', 'Mineração'),
('ITUB4', 'Itaú Unibanco PN', 'Financeiro'),
('MGLU3', 'Magazine Luiza ON', 'Varejo'),
('WEGE3', 'WEG ON', 'Industrial');

-- UPSERT: se o pregão já existir, atualiza em vez de duplicar (ideal para reprocessar arquivos).
INSERT INTO cotacoes (ticker, data, abertura, maxima, minima, fechamento, volume)
VALUES ('PETR4', '2025-03-10', 38.10, 38.90, 37.85, 38.60, 42311000)
ON CONFLICT(ticker, data) DO UPDATE SET
    fechamento = excluded.fechamento, -- dispara ao violar a PRIMARY KEY (ticker,data)
    volume = excluded.volume; -- 'excluded' = a linha que tentou entrar
```

4.2 Carga em massa a partir do Python (executemany)

PYTHON

```
linhas = [
    ("PETR4", "2025-03-10", 38.10, 38.90, 37.85, 38.60, 42311000),
    ("PETR4", "2025-03-11", 38.62, 39.40, 38.50, 39.22, 39120000),
]

con.executemany(
    """INSERT OR REPLACE INTO cotacoes
        (ticker, data, abertura, maxima, minima, fechamento, volume)
        VALUES (?, ?, ?, ?, ?, ?, ?)""",
    linhas,
)
con.commit()

# envia todas as tuplas num único lote
# '?' = placeholder, protege de SQL Injection
# grava tudo de uma vez: MUITO mais rápido
```

4.3 Importando um CSV inteiro com pandas

PYTHON

```
df = pd.read_csv("cotacoes.csv", parse_dates=["data"]) # lê o arquivo e converte a coluna de data
df["data"] = df["data"].dt.strftime("%Y-%m-%d") # padroniza no formato ISO exigido pelo banco

df.to_sql(
    "cotacoes",
    con,
    if_exists="append",
    index=False,
    chunksize=5000
)

# despeja o DataFrame direto numa tabela
# nome da tabela de destino
# conexão sqlite3 aberta
# 'append' acrescenta; 'replace' apagaria a tabela antes
# não gravar o índice do DataFrame como coluna
# envia de 5.000 em 5.000 linhas (controla o uso de memória)
```

Dica de ouro

Antes de uma carga grande use `con.execute('PRAGMA synchronous=OFF')` e envolva tudo em uma única transação. A importação pode ficar **10x** mais rápida.

5. SQL essencial para finanças

Estas consultas aparecem repetidamente nos capítulos de gráficos. Vale ler com calma — dominando estas seis, você monta praticamente qualquer visualização.

5.1 Último preço de cada ativo

```
SQL
SELECT c.ticker,          -- código do papel
       c.data,            -- data do pregão mais recente encontrado
       c.fechamento      -- preço que será usado nos cálculos de mercado
FROM cotacoes c
WHERE c.data = (          -- subconsulta correlacionada: filtra a data máxima...
    SELECT MAX(c2.data)   -- ...daquele mesmo ticker
    FROM cotacoes c2
    WHERE c2.ticker = c.ticker
)
ORDER BY c.ticker;       -- saída em ordem alfabética, boa para legendas
```

5.2 Agrupando por mês com substr()

```
SQL
SELECT substr(data, 1, 7) AS mes,      -- pega 'AAAA-MM' dos 7 primeiros caracteres da data ISO
       ROUND(AVG(fechamento), 2) AS preco_medio, -- média dos fechamentos do mês
       MAX(maxima) AS topo,           -- maior cotação intradiária do mês
       MIN(minima) AS fundo,          -- menor cotação intradiária do mês
       SUM(volume) AS volume_total    -- volume acumulado no período
FROM cotacoes
WHERE ticker = 'PETR4'                -- restringe a um único ativo
GROUP BY mes                          -- uma linha por mês
ORDER BY mes;                        -- cronológico: essencial para o eixo X do gráfico
```

5.3 Variação diária com LAG (window function)

```
SQL
SELECT data,
       fechamento,
       LAG(fechamento) OVER (ORDER BY data) AS anterior, -- valor da LINHA ANTERIOR
       ROUND(
         (fechamento / LAG(fechamento) OVER (ORDER BY data) - 1) * 100
       , 2) AS variacao_pct -- retorno diário em porcentagem
FROM cotacoes
WHERE ticker = 'VALE3'
ORDER BY data;
```

Nota

Window functions (LAG, LEAD, AVG() OVER, SUM() OVER) existem no SQLite desde a versão 3.25. São a ferramenta mais poderosa para séries temporais: calculam olhando linhas vizinhas **sem** precisar de auto-join.

5.4 Média móvel de 20 períodos

```
SQL
SELECT data,
       fechamento,
       ROUND(AVG(fechamento) OVER (
         ORDER BY data -- define a ordem da janela
         ROWS BETWEEN 19 PRECEDING AND CURRENT ROW -- 19 anteriores + a atual = 20 dias
       ), 2) AS mm20
FROM cotacoes
WHERE ticker = 'ITUB4'
ORDER BY data;
```

5.5 CTE: encadeando etapas legíveis

SQL

```
WITH diario AS ( -- etapa 1: calcula o retorno de cada dia
    SELECT ticker, data, fechamento,
           fechamento / LAG(fechamento) OVER (PARTITION BY ticker ORDER BY data) - 1 AS ret
    FROM cotacoes
),
resumo AS ( -- etapa 2: consolida por ativo
    SELECT ticker,
           COUNT(*) AS pregoes,
           ROUND(AVG(ret) * 100, 3) AS ret_medio_pct,
           ROUND(MAX(fechamento), 2) AS maxima_periodo
    FROM diario
    WHERE ret IS NOT NULL -- descarta o primeiro dia (não tem anterior)
    GROUP BY ticker
)
SELECT * FROM resumo ORDER BY ret_medio_pct DESC; -- ranking do melhor ao pior
```

5.6 JOIN com a carteira: valor de mercado

SQL

```
SELECT a.ticker,
       a.nome,
       a.setor,
       k.quantidade,
       k.preco_medio,
       u.fechamento AS preco_atual,
       ROUND(k.quantidade * u.fechamento, 2) AS valor_mercado,
       ROUND(k.quantidade * (u.fechamento - k.preco_medio), 2) AS lucro
FROM carteira k
JOIN ativos a ON a.ticker = k.ticker -- traz nome e setor
JOIN ( -- subconsulta com o último preço por ativo
    SELECT ticker, fechamento
    FROM cotacoes c
    WHERE data = (SELECT MAX(data) FROM cotacoes WHERE ticker = c.ticker)
) u ON u.ticker = k.ticker
ORDER BY valor_mercado DESC;
```


6. A ponte SQL → pandas → matplotlib

O padrão de 5 linhas que se repete em todos os gráficos.

PYTHON

```
# ---- 1) A CONSULTA: o SQL entrega apenas o que o gráfico precisa ----
sql = """
SELECT data, fechamento
FROM cotacoes
WHERE ticker = ?          -- parâmetro: nunca concatene texto na consulta!
ORDER BY data
"""

# ---- 2) A LEITURA: SQL vira DataFrame ----
df = pd.read_sql_query(sql, con, params=("PETR4",)) # params fecha a porta do SQL Injection

# ---- 3) O AJUSTE: tipos e índice ----
df["data"] = pd.to_datetime(df["data"]) # texto ISO -> datetime (o eixo X entende datas)
df = df.set_index("data")              # datas viram índice: facilita resample e rolling

# ---- 4) O DESENHO ----
fig, ax = plt.subplots(figsize=(9, 3.5)) # 'fig' é a folha; 'ax' é a área do gráfico
ax.plot(df.index, df["fechamento"])     # série no eixo X (datas) e Y (preços)

# ---- 5) A SAÍDA ----
fig.tight_layout()                     # ajusta margens para não cortar rótulos
fig.savefig("grafico.png", dpi=150)    # dpi 150 = boa qualidade para relatório/PDF
plt.close(fig)                         # libera memória (obrigatório em laços!)
```

Nota

Memorize a assinatura `fig, ax = plt.subplots()`. Tudo que é **página** (salvar, tamanho) é método de `fig`; tudo que é **conteúdo** (linhas, títulos, eixos) é método de `ax`.

7. Gráfico de linha: histórico de preços

O gráfico mais básico e mais usado do mercado financeiro.

A série de fechamentos mostra a trajetória do preço no tempo. É o ponto de partida de qualquer análise: tendência, suportes, resistências e rupturas ficam visíveis de imediato.

A consulta SQL

SQL

```
SELECT data,          -- eixo X do gráfico (texto ISO, já ordenável)
       fechamento    -- eixo Y: preço de fechamento em reais
FROM cotacoes
WHERE ticker = 'PETR4' -- um ativo por vez mantém a escala legível
ORDER BY data;        -- ORDER BY é OBRIGATÓRIO: sem ele a linha vira um rabisco
```

O código Python comentado

PYTHON

```
import sqlite3, pandas as pd, matplotlib.pyplot as plt

con = sqlite3.connect("financas.db")          # abre o arquivo do banco

sql = """SELECT data, fechamento
      FROM cotacoes
      WHERE ticker = ?
      ORDER BY data"""
df = pd.read_sql_query(sql, con, params=("PETR4",)) # executa e recebe um DataFrame
df["data"] = pd.to_datetime(df["data"])           # converte texto -> datetime

fig, ax = plt.subplots(figsize=(9, 3.5))          # folha de 9x3,5 polegadas
ax.plot(df["data"], df["fechamento"],           # desenha a série temporal
        color="#0f4c81",                         # azul corporativo
        linewidth=1.3)                             # linha fina: evita "borrar" 500 pontos

ax.set_title("PETR4 — Fechamento diário", fontweight="bold") # título em negrito
ax.set_ylabel("R$")                                   # unidade no eixo Y
ax.grid(alpha=0.25)                                   # grade discreta ajuda a ler valores
ax.spines[["top", "right"]].set_visible(False)        # remove bordas superiores: visual limpo

fig.autofmt_xdate()                                   # inclina as datas para não sobrepor
fig.tight_layout()
fig.savefig("linha_petr4.png", dpi=150)
```

Resultado



Figura 7.1 — Série de fechamentos diários de PETR4.

Repare em três decisões de estilo que valem para todos os gráficos: linha fina (séries longas com linha grossa perdem detalhe), grade com transparência baixa (informa sem competir com os dados) e remoção das bordas superior e direita (elimina ruído visual).

Se a série tiver muitos anos, agregue antes no SQL — por exemplo, `AVG(fechamento) GROUP BY substr(data,1,7)` para um ponto por mês. Menos pontos, leitura mais clara.

8. Comparando ativos (base 100)

Como comparar papéis de preços muito diferentes.

Comparar PETR4 (R\$ 36) com VALE3 (R\$ 68) no mesmo eixo esconde a informação que interessa: **quem rendeu mais**. A solução clássica é normalizar tudo para a base 100 no primeiro dia — a partir daí a distância vertical entre as linhas é a diferença de desempenho em porcentagem.

A consulta SQL

SQL

```
-- Traz todos os ativos de uma vez; a normalização é feita no pandas.
SELECT ticker,      -- servirá para "pivotar": um ativo por coluna
       data,        -- eixo X comum a todas as séries
       fechamento  -- valor bruto que será convertido em base 100
FROM cotacoes
ORDER BY data;      -- ordenação global garante o alinhamento das datas
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query("SELECT ticker, data, fechamento FROM cotacoes ORDER BY data", con)
df["data"] = pd.to_datetime(df["data"])

# pivot: linhas = datas, colunas = tickers, valores = preços
tabela = df.pivot(index="data", columns="ticker", values="fechamento")

# normalização base 100: cada coluna dividida pelo SEU primeiro valor
base100 = tabela / tabela.iloc[0] * 100      # iloc[0] = primeira linha (data mais antiga)

fig, ax = plt.subplots(figsize=(9, 4))
for ticker in base100.columns:
    ax.plot(base100.index, base100[ticker], # série normalizada
            linewidth=1.2, label=ticker)   # label alimenta a legenda automaticamente

ax.axhline(100, color="#94a3b8", linestyle="--", linewidth=0.8) # linha de referência do ponto zero
ax.legend(ncol=5, fontsize=8)                                     # legenda em 5 colunas: ocupa pouca altura
ax.set_title("Desempenho relativo (base 100)", fontweight="bold")
ax.set_ylabel("índice")
fig.tight_layout(); fig.savefig("base100.png", dpi=150)
```

Resultado

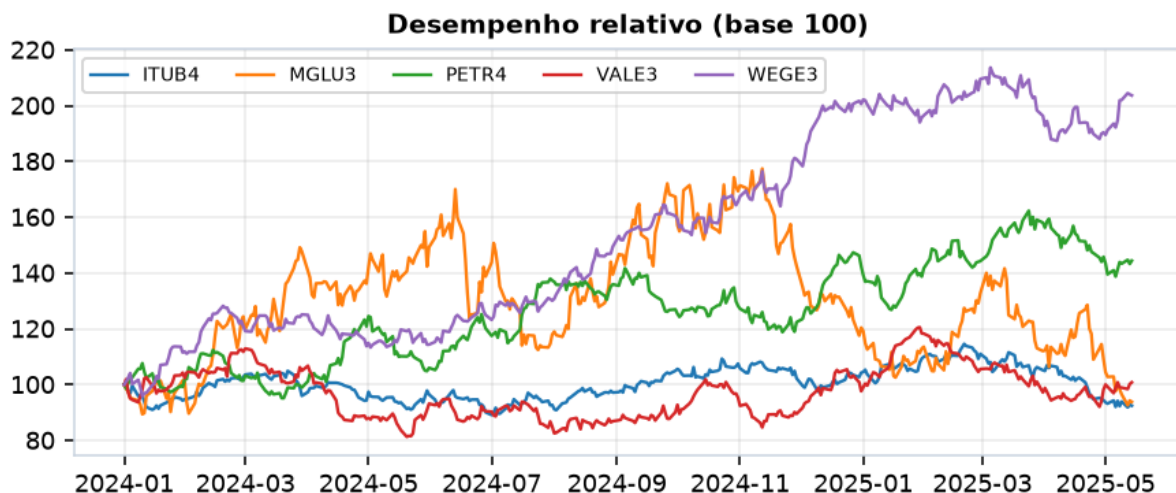


Figura 8.1 — Todas as séries partem de 100; ler o eixo Y é ler o retorno acumulado.

Um valor de 130 significa +30% desde o início; 85 significa –15%. Como todas as curvas partem do mesmo ponto, o olho compara trajetórias sem se distrair com o preço nominal.

O mesmo truque serve para comparar sua carteira contra o Ibovespa ou o CDI: basta trazer o índice como mais uma linha da tabela cotacoes.

9. Candlestick + volume

O gráfico clássico do analista técnico, desenhado do zero.

Cada candle resume um pregão inteiro com quatro números (OHLC): a linha vertical fina liga a mínima à máxima e o corpo retangular liga a abertura ao fechamento. Verde quando fechou acima da abertura, vermelho quando fechou abaixo. O volume embaixo confirma (ou desmente) a força do movimento.

A consulta SQL

SQL

```
-- Últimos 45 pregões, do mais antigo para o mais novo.
SELECT data, abertura, maxima, minima, fechamento, volume
FROM (
    SELECT *
    FROM cotacoes
    WHERE ticker = 'VALE3'
    ORDER BY data DESC          -- pega os mais recentes...
    LIMIT 45                   -- ...limitando a 45 linhas
)
ORDER BY data ASC;           -- reinverte: o gráfico precisa da ordem cronológica
```

O código Python comentado

PYTHON

```
d = pd.read_sql_query(sql, con).reset_index(drop=True) # índice 0..44 vira a posição no eixo X

fig, (ax1, ax2) = plt.subplots(
    2, 1,
    figsize=(9, 5), sharex=True,          # duas linhas de gráficos, uma coluna
    gridspec_kw={'height_ratios': [3, 1]} # eixo X compartilhado: os dias ficam alinhados
)
# o preço ocupa 3/4 e o volume 1/4 da altura

for i, r in d.iterrows():
    # percorre pregão a pregão
    alta = r.fechamento >= r.abertura    # True = candle de alta
    cor = "#0f8a5f" if alta else "#c0392b" # verde para alta, vermelho para baixa

    ax1.vlines(i, r.minima, r.maxima,      # o "pavio": linha da mínima até a máxima
               color=cor, linewidth=0.8)

    corpo_base = min(r.abertura, r.fechamento) # base do retângulo
    corpo_altura = max(abs(r.fechamento - r.abertura), 0.01) # altura (mínimo p/ doji visível)
    ax1.add_patch(plt.Rectangle((i - 0.3, corpo_base), # canto inferior esquerdo
                                0.6, corpo_altura, color=cor)) # largura 0,6 e altura calculada

    ax2.bar(i, r.volume, color=cor, width=0.6) # barra de volume com a mesma cor

ax1.set_xlim(-1, len(d)) # folga nas laterais para não cortar candles
ax1.set_title("VALE3 - Candlestick + Volume", fontweight="bold")
ax1.set_ylabel("R$"); ax2.set_ylabel("Volume")
ax2.set_xticks([]) # esconde os índices numéricos do eixo X
fig.tight_layout(); fig.savefig("candle.png", dpi=150)
```

Resultado

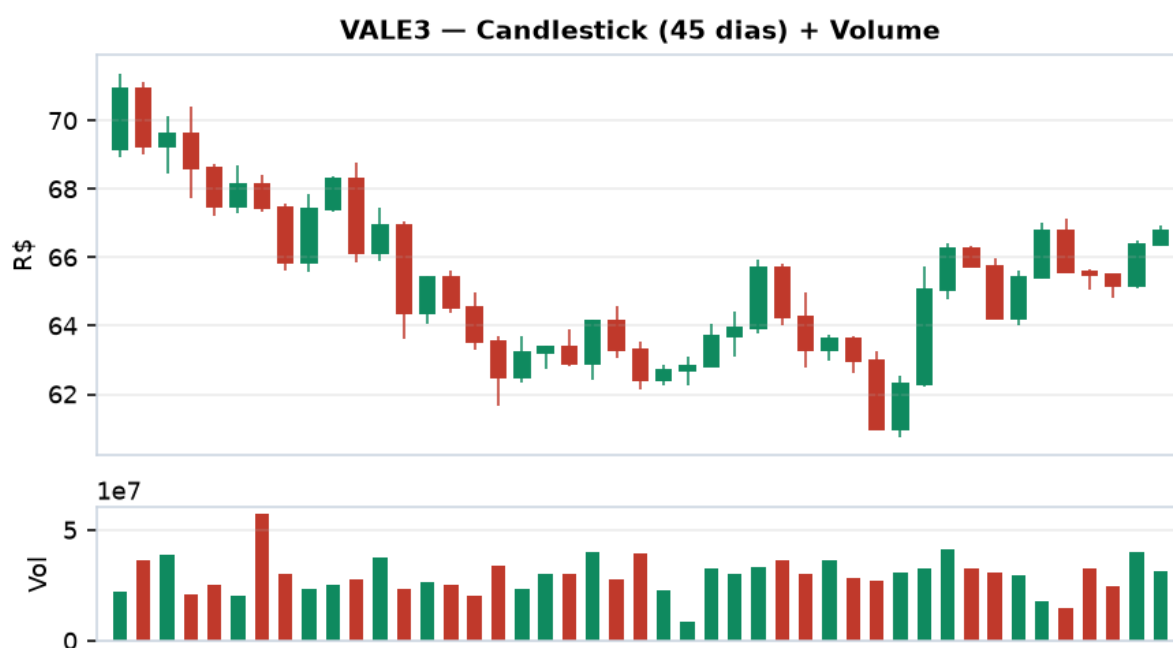


Figura 9.1 — Candlestick de 45 pregões com o volume correspondente abaixo.

Por que usar a **posição** (0, 1, 2...) no eixo X em vez da data? Porque o mercado não abre fins de semana e feriados: usando datas reais o matplotlib deixaria buracos vazios entre as semanas. Numerando os pregões, os candles ficam colados, exatamente como nas plataformas profissionais.

Regra de leitura: alta com volume crescente indica convicção; alta com volume minguando costuma anteceder correção.

10. Médias móveis com window functions

Deixe o SQLite calcular o indicador para você.

A média móvel suaviza o ruído diário e revela a tendência. O cruzamento da MM20 (curta) acima da MM50 (longa) é o clássico sinal de compra; o cruzamento para baixo, de venda. O ponto alto deste exemplo é que **o cálculo acontece no banco**, não no Python.

A consulta SQL

SQL

```
SELECT data,
       fechamento,
       ROUND(AVG(fechamento) OVER (
         ORDER BY data
         ROWS BETWEEN 19 PRECEDING AND CURRENT ROW -- janela de 20 pregões
       ), 2) AS mm20,
       ROUND(AVG(fechamento) OVER (
         ORDER BY data
         ROWS BETWEEN 49 PRECEDING AND CURRENT ROW -- janela de 50 pregões
       ), 2) AS mm50
FROM cotacoes
WHERE ticker = 'ITUB4'
ORDER BY data;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con) # o banco já devolve mm20 e mm50 prontas
df["data"] = pd.to_datetime(df["data"])

fig, ax = plt.subplots(figsize=(9, 3.8))
ax.plot(df.data, df.fechamento, color="#94a3b8", lw=0.9, label="Fechamento") # cinza: pano de fundo
ax.plot(df.data, df.mm20, color="#0f4c81", lw=1.5, label="MM20") # azul: média curta
ax.plot(df.data, df.mm50, color="#e67e22", lw=1.5, label="MM50") # laranja: média longa

# Alternativa 100% pandas (útil quando o SQL não tem window functions):
# df["mm20"] = df["fechamento"].rolling(window=20).mean() # janela deslizante de 20 linhas
# df["mm50"] = df["fechamento"].rolling(window=50).mean()

ax.legend(fontsize=8); ax.set_title("ITUB4 — Médias móveis", fontweight="bold")
fig.tight_layout(); fig.savefig("mm.png", dpi=150)
```

Resultado

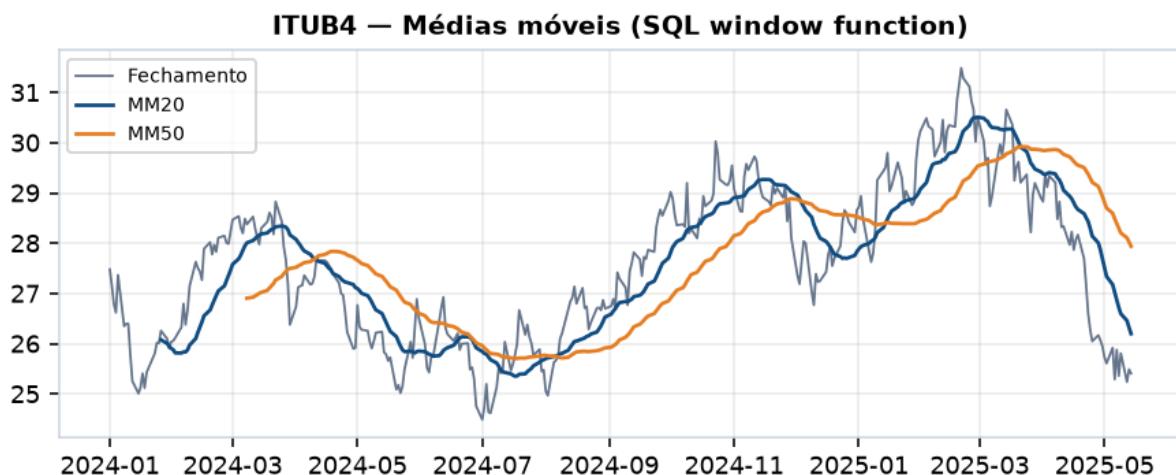


Figura 10.1 — Preço em cinza e as duas médias destacadas em azul e laranja.

Cuidado com um detalhe: `ROWS BETWEEN 19 PRECEDING` já retorna valores nas primeiras 19 linhas (calculando com menos dias). Se quiser o comportamento do pandas, que devolve NaN até completar a janela, adicione `CASE WHEN ROW_NUMBER() OVER (ORDER BY data) < 20 THEN NULL ELSE ... END`.

Hierarquia visual importa: a série principal fica em cinza claro e os indicadores em cores fortes, porque é neles que o olho deve pousar.

11. Retorno mensal em barras

Verde para ganho, vermelho para perda.

Barras coloridas por sinal são a forma mais direta de mostrar desempenho período a período. O cálculo compara o primeiro e o último fechamento de cada mês.

A consulta SQL

SQL

```
WITH marcados AS (
    SELECT substr(data,1,7) AS mes,
           data, fechamento,
           ROW_NUMBER() OVER (PARTITION BY substr(data,1,7) ORDER BY data ASC) AS rn_ini,
           ROW_NUMBER() OVER (PARTITION BY substr(data,1,7) ORDER BY data DESC) AS rn_fim
    FROM cotacoes
    WHERE ticker = 'PETR4'
)
SELECT mes,
       MAX(CASE WHEN rn_ini = 1 THEN fechamento END) AS abre, -- fechamento do 1º dia útil
       MAX(CASE WHEN rn_fim = 1 THEN fechamento END) AS fecha, -- fechamento do último dia útil
       ROUND((MAX(CASE WHEN rn_fim = 1 THEN fechamento END)
              / MAX(CASE WHEN rn_ini = 1 THEN fechamento END) - 1) * 100, 2) AS retorno_pct
FROM marcados
GROUP BY mes
ORDER BY mes;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

cores = ["#0f8a5f" if v >= 0 else "#c0392b"
         for v in df["retorno_pct"]] # lista de cores, uma por barra
                                     # verde se positivo, vermelho se negativo

fig, ax = plt.subplots(figsize=(9, 3.5))
ax.bar(df["mes"], df["retorno_pct"], color=cores) # barras verticais com cor condicional
ax.axhline(0, color="#334155", linewidth=0.8) # linha do zero separa ganho de perda
ax.set_xticklabels(df["mes"], rotation=60, fontsize=7) # meses inclinados para caber

for x, v in enumerate(df["retorno_pct"]):
    ax.text(x, v + (0.4 if v >= 0 else -0.9), # rótulo numérico em cima de cada barra
            f"{v:.1f}%", ha="center", fontsize=6) # desloca acima ou abaixo conforme o sinal

ax.set_title("PETR4 — Retorno mensal (%)", fontweight="bold")
fig.tight_layout(); fig.savefig("ret_mensal.png", dpi=150)
```

Resultado

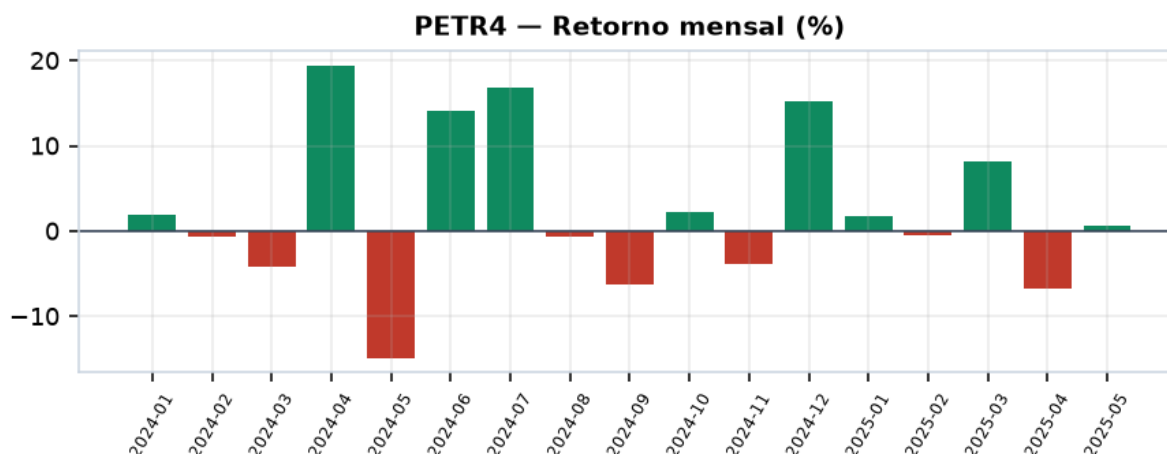


Figura 11.1 — Meses positivos em verde, negativos em vermelho.

O padrão de cor **verde = ganho / vermelho = perda** é universal no mercado brasileiro e dispensa legenda. Sempre desenhe a linha do zero: sem ela, barras pequenas de sinais opostos ficam ambíguas.

12. Pizza: composição da carteira

Onde o dinheiro realmente está.

A pizza responde a uma pergunta única: qual a fatia de cada ativo no patrimônio? Use-a apenas com poucas categorias (até 6) e quando as partes somam um todo.

A consulta SQL

SQL

```
SELECT k.ticker,
       ROUND(k.quantidade * u.fechamento, 2) AS valor_mercado  -- posição avaliada a preço de hoje
FROM carteira k
JOIN (SELECT ticker, fechamento                                -- último preço de cada papel
      FROM cotacoes c
      WHERE data = (SELECT MAX(data) FROM cotacoes WHERE ticker = c.ticker)) u
ON u.ticker = k.ticker
ORDER BY valor_mercado DESC;                                   -- maior fatia primeiro
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

fig, ax = plt.subplots(figsize=(6, 4.5))
fatias, textos, pct = ax.pie(
    df["valor_mercado"],          # tamanho de cada fatia
    labels=df["ticker"],         # rótulo externo
    autopct="%1.1f%%",          # percentual dentro da fatia, 1 casa decimal
    startangle=90,               # começa no topo: leitura mais natural
    counterclock=False,          # sentido horário, como um relógio
    explode=[0.04] + [0] * (len(df) - 1), # destaca levemente a maior posição
    colors=["#0f4c81", "#0f8a5f", "#e67e22", "#c0392b", "#64748b"],
    wedgeprops={"edgecolor": "white", "linewidth": 1.5} # borda branca separa as fatias
)
for t in pct: t.set_color("white") # percentual branco contrasta com o fundo escuro

total = df["valor_mercado"].sum() # patrimônio total, para o subtítulo
ax.set_title(f"Carteira - R$ {total:,.2f}".replace(",", "."), fontweight="bold")
fig.tight_layout(); fig.savefig("pizza.png", dpi=150)
```

Resultado

Composição da carteira (valor de mercado)

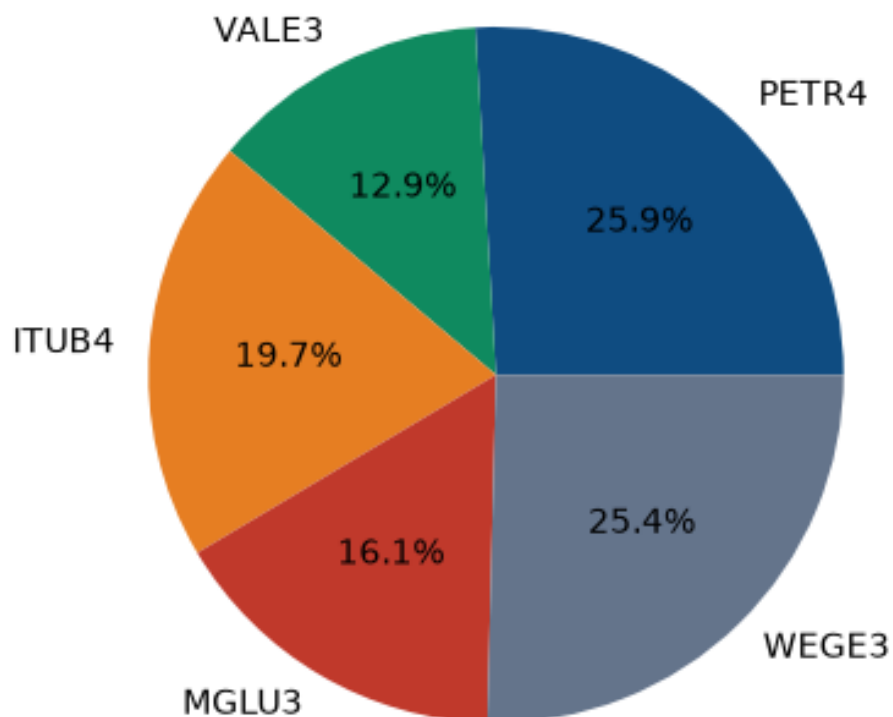


Figura 12.1 — Participação de cada ativo no valor de mercado da carteira.

Com mais de 6 ativos, troque a pizza por barras horizontais ordenadas: o olho compara comprimentos muito melhor do que ângulos. Uma variação elegante é o donut: acrescente `wedgeprops={'width':0.42}` e escreva o total no centro com `ax.text(0,0,...)`.

13. Lucro/prejuízo por ativo

Barras horizontais divergentes.

Quando os valores têm sinais opostos e os rótulos são textos, barras horizontais são superiores: os nomes ficam legíveis e o zero central separa naturalmente ganhadores de perdedores.

A consulta SQL

SQL

```
SELECT k.ticker,
       k.quantidade,
       k.preco_medio,
       u.fechamento AS preco_atual,
       ROUND(k.quantidade * (u.fechamento - k.preco_medio), 2) AS lucro_prejuizo,
       ROUND((u.fechamento / k.preco_medio - 1) * 100, 2) AS rentabilidade_pct
FROM carteira k
JOIN (SELECT ticker, fechamento FROM cotacoes c
      WHERE data = (SELECT MAX(data) FROM cotacoes WHERE ticker = c.ticker)) u
ON u.ticker = k.ticker
ORDER BY lucro_prejuizo;      -- crescente: prejuízos embaixo, lucros em cima no gráfico
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

cores = ["#0f8a5f" if v >= 0 else "#c0392b" for v in df["lucro_prejuizo"]]

fig, ax = plt.subplots(figsize=(9, 3.2))
barras = ax.barh(df["ticker"], df["lucro_prejuizo"], color=cores) # barh = barras horizontais
ax.axvline(0, color="#334155", linewidth=0.8) # eixo do zero na vertical

def reais(x, pos):
    return f"R$ {x:,.0f}".replace(",", ".") # formatador de moeda brasileira
    # troca a vírgula do milhar por ponto
from matplotlib.ticker import FuncFormatter
ax.xaxis.set_major_formatter(FuncFormatter(reais)) # aplica no eixo X

for b, v in zip(barras, df["lucro_prejuizo"]):
    ax.text(v + (60 if v >= 0 else -60), # rótulo ao final de cada barra
            b.get_y() + b.get_height()/2, # empurra para fora da barra
            f"R$ {v:,.0f}".replace(",", "."), # centraliza verticalmente
            va="center", ha="left" if v >= 0 else "right", fontsize=7)

ax.set_title("Lucro / prejuízo não realizado", fontweight="bold")
fig.tight_layout(); fig.savefig("pl.png", dpi=150)
```

Resultado

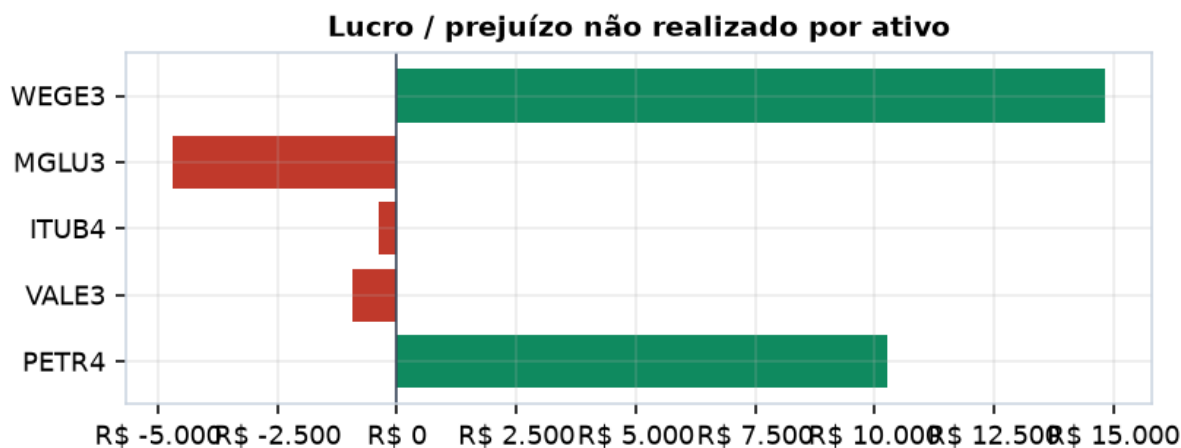


Figura 13.1 — Resultado em reais de cada posição, comparando preço médio e cotação atual.

FuncFormatter é o recurso que transforma 12000 em R\$ 12.000 no eixo. Formatar valores no padrão brasileiro é o detalhe que separa um gráfico amador de um relatório profissional.

14. Despesas mensais empilhadas

Total e composição no mesmo desenho.

A barra empilhada mostra duas informações de uma vez: a **altura total** (quanto se gastou no mês) e a **composição** (quanto foi de cada categoria).

A consulta SQL

SQL

```
SELECT substr(data, 1, 7) AS mes,      -- agrupa por ano-mês
       categoria,                    -- vira uma "camada" da pilha
       ROUND(SUM(valor), 2) AS total  -- soma dos gastos daquela categoria no mês
FROM despesas
GROUP BY mes, categoria              -- uma linha por combinação mês+categoria
ORDER BY mes, categoria;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

tabela = df.pivot(index="mes", columns="categoria", values="total").fillna(0)
# resultado: linhas = meses, colunas = categorias, células = valor (0 quando não houve gasto)

import numpy as np
fig, ax = plt.subplots(figsize=(9, 4))
base = np.zeros(len(tabela))
for categoria in tabela.columns:
    ax.bar(tabela.index, tabela[categoria],
           bottom=base,
           label=categoria)
    base += tabela[categoria].values
# acumulator da altura já empilhada
# empilha uma categoria por vez
# começa onde a camada anterior terminou
# atualiza o acumulador

ax.legend(fontsize=7, ncol=3)
ax.set_xticks(range(len(tabela.index)))
ax.set_xticklabels(tabela.index, rotation=60, fontsize=7)
ax.set_title("Despesas mensais por categoria", fontweight="bold")
fig.tight_layout(); fig.savefig("despesas.png", dpi=150)
```

Resultado

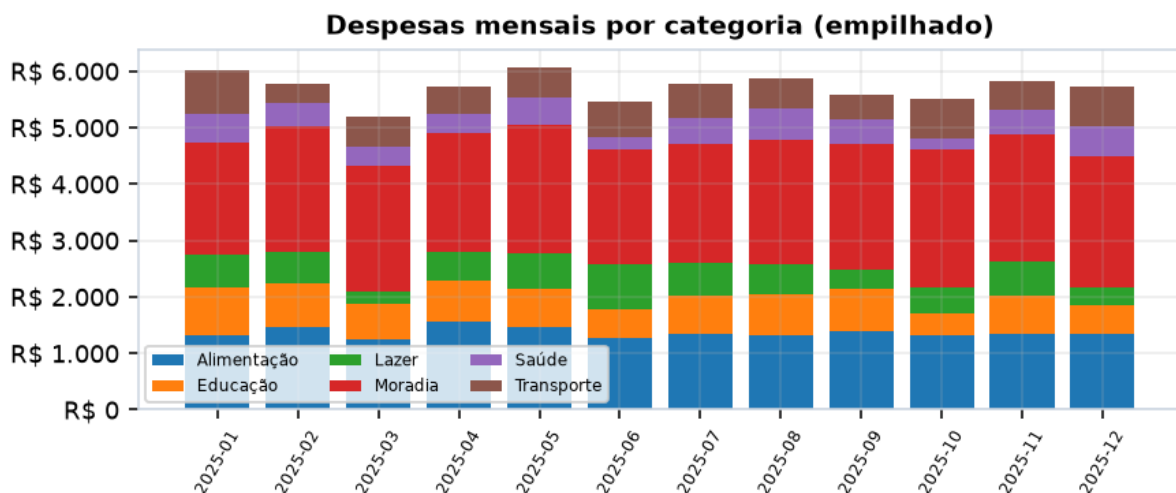


Figura 14.1 — Cada cor é uma categoria; a altura total é o gasto do mês.

Limitação honesta do empilhado: só a camada de baixo tem uma linha de base reta, então comparar as camadas superiores entre meses é difícil. Se a pergunta for “a categoria Lazer

cresceu?”, prefira linhas separadas ou pequenos múltiplos.

Para ver **proporção** em vez de valor absoluto, normalize: `tabela = tabela.div(tabela.sum(axis=1), axis=0) * 100`.

15. Matriz de correlação (heatmap)

Diversificação de verdade se mede aqui.

Correlação próxima de +1 significa que os ativos sobem e caem juntos — logo, diversificam pouco. Próxima de 0, os movimentos são independentes. Negativa, um protege o outro. O heatmap mostra tudo de relance.

A consulta SQL

SQL

```
-- O SQL entrega os retornos diários; a correlação em si é feita no pandas.
SELECT ticker,
       data,
       fechamento / LAG(fechamento) OVER (PARTITION BY ticker ORDER BY data) - 1 AS retorno
FROM cotacoes
ORDER BY data;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con).dropna()          # remove o 1º dia de cada ativo (sem anterior)
tabela = df.pivot(index="data", columns="ticker", values="retorno")

matriz = tabela.corr()                             # correlação de Pearson entre TODAS as colunas

fig, ax = plt.subplots(figsize=(5.5, 4.5))
im = ax.imshow(matriz, cmap="RdYlGn", vmin=-1, vmax=1) # escala fixa -1..+1 evita enganos

ax.set_xticks(range(len(matriz))); ax.set_xticklabels(matriz.columns, rotation=45)
ax.set_yticks(range(len(matriz))); ax.set_yticklabels(matriz.columns)
ax.grid(False)                                     # grade atrapalha em heatmap

for i in range(len(matriz)):                        # escreve o número dentro de cada célula
    for j in range(len(matriz)):
        ax.text(j, i, f"{matriz.iloc[i, j]:.2f}", ha="center", va="center", fontsize=7)

fig.colorbar(im, fraction=0.046)                   # barra de cores explicando a escala
ax.set_title("Correlação dos retornos diários", fontweight="bold")
fig.tight_layout(); fig.savefig("corr.png", dpi=150)
```

Resultado

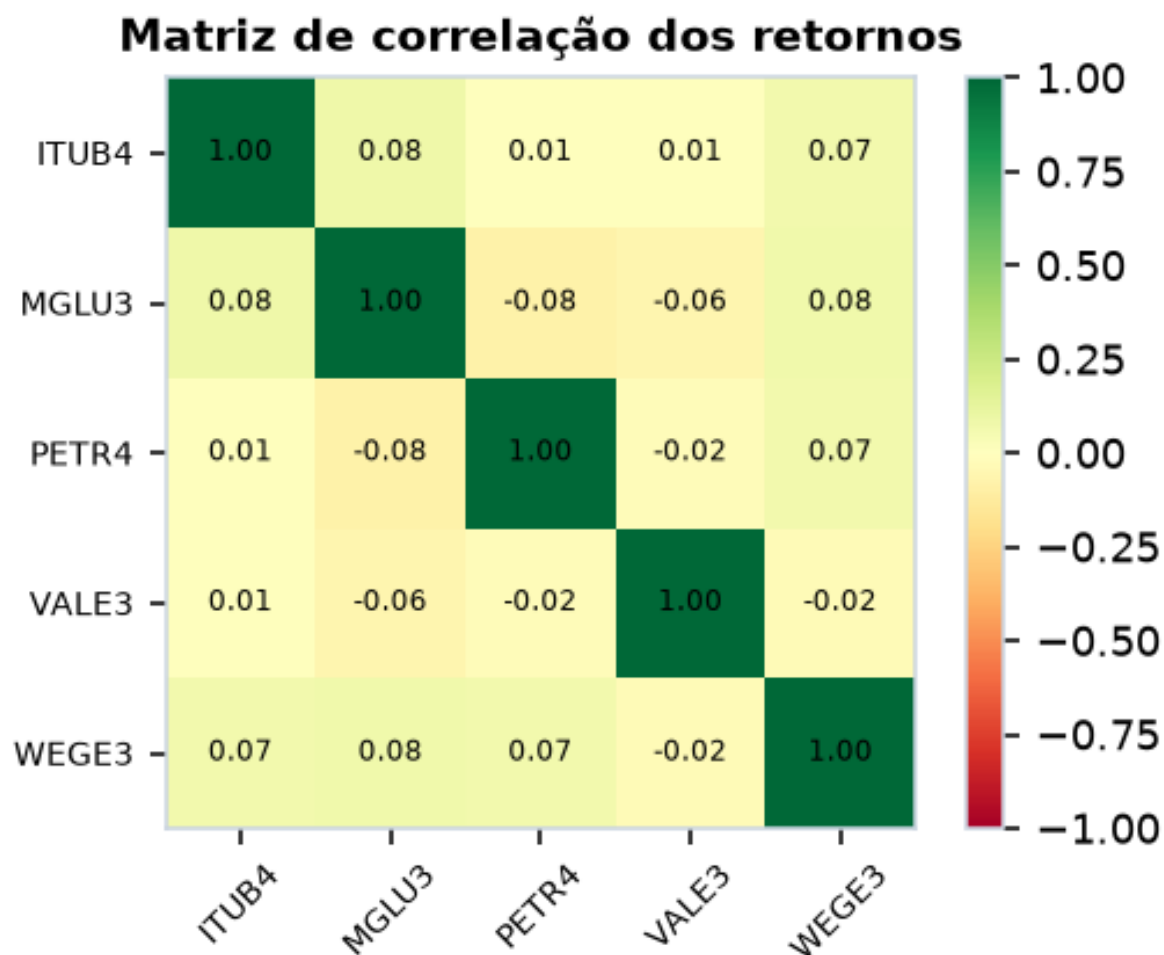


Figura 15.1 — Diagonal sempre 1,00 (o ativo é perfeitamente correlacionado consigo mesmo).

Fixar `vmin=-1`, `vmax=1` é essencial: sem isso o matplotlib normaliza a escala pelos dados e uma correlação de 0,3 pode aparecer com a mesma cor forte de uma de 0,95.

Sempre correlacione **retornos**, nunca preços. Séries de preço com tendência produzem correlações artificialmente altas.

16. Drawdown

Quanto o investimento caiu desde o topo.

O drawdown mede a queda percentual em relação ao maior valor já atingido. É a métrica de risco que o investidor sente na pele — mais do que a volatilidade estatística.

A consulta SQL

SQL

```
WITH serie AS (
  SELECT data,
         fechamento,
         MAX(fechamento) OVER (ORDER BY data
                                ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS pico
  FROM cotacoes
  WHERE ticker = 'MGLU3'
)
SELECT data,
       fechamento,
       pico,
       ROUND((fechamento / pico - 1) * 100, 2) AS drawdown_pct -- sempre <= 0
FROM serie
ORDER BY data;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)
df["data"] = pd.to_datetime(df["data"])

# Equivalente em pandas, caso prefira calcular fora do banco:
# df["pico"] = df["fechamento"].cummax() # máximo acumulado
# df["drawdown_pct"] = (df["fechamento"]/df["pico"] - 1)*100 # queda em relação ao pico

fig, ax = plt.subplots(figsize=(9, 3.2))
ax.fill_between(df.data, df.drawdown_pct, 0, # área entre a curva e a linha do zero
               color="#c0392b", alpha=0.45) # vermelho translúcido = "área de dor"
ax.plot(df.data, df.drawdown_pct, color="#c0392b", lw=0.9)

pior = df["drawdown_pct"].min() # o pior momento da série
ax.annotate(f"máx. drawdown {pior:.1f}%", # anotação com seta apontando o fundo
           xy=(df.loc[df.drawdown_pct.idxmin(), "data"], pior),
           xytext=(0.35, 0.15), textcoords="axes fraction",
           arrowprops=dict(arrowstyle="->", color="#334155"), fontsize=8)

ax.set_title("MGLU3 - Drawdown (%)", fontweight="bold")
fig.tight_layout(); fig.savefig("drawdown.png", dpi=150)
```

Resultado

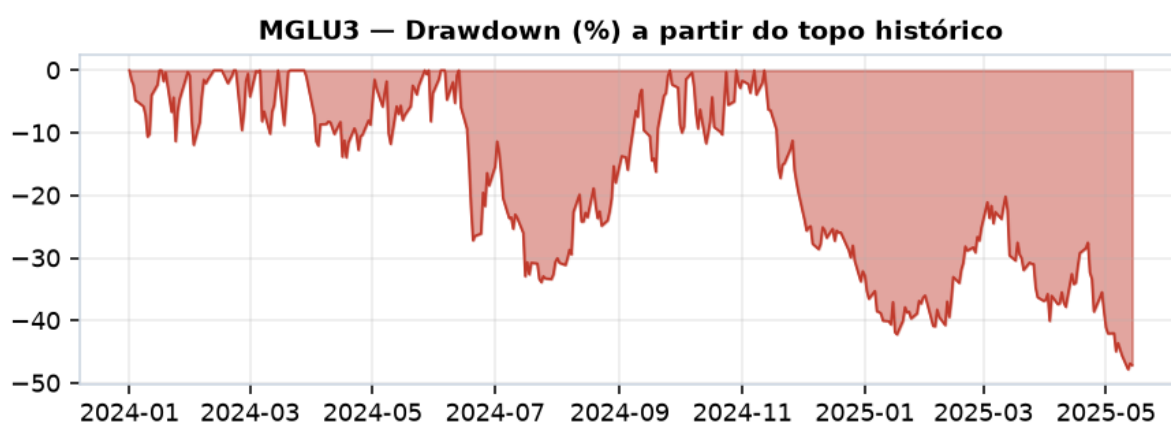


Figura 16.1 — Área vermelha: todo o período em que o preço esteve abaixo do topo anterior.

O gráfico responde a duas perguntas de risco: **qual foi a maior queda** (profundidade) e **quanto tempo levou para recuperar** (largura da área). Um fundo com -15% recuperado em 2 meses é bem diferente de um com -15% que durou 3 anos.

17. Histograma de retornos

A forma da volatilidade.

O histograma conta quantos dias caíram em cada faixa de retorno. Séries financeiras têm 'caudas gordas': eventos extremos acontecem com muito mais frequência do que a curva normal prevê.

A consulta SQL

SQL

```
SELECT ROUND(
    (fechamento / LAG(fechamento) OVER (ORDER BY data) - 1) * 100
    , 4) AS retorno_pct      -- retorno diário em %, com 4 casas para não perder precisão
FROM cotacoes
WHERE ticker = 'PETR4'
ORDER BY data;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con).dropna()          # descarta o primeiro dia (retorno nulo)
r = df["retorno_pct"]

fig, ax = plt.subplots(figsize=(9, 3.4))
ax.hist(r, bins=40, color="#0f4c81", alpha=0.85, edgecolor="white")
#      ^bins=40: número de faixas. Poucas escondem detalhe, muitas viram ruído.

ax.axvline(r.mean(), color="#e67e22", lw=1.5,      # média em laranja
           label=f"média {r.mean():.2f}%")
ax.axvline(r.mean() - 2*r.std(), color="#c0392b", ls="--", lw=1, # -2 desvios padrão
           label=f"-2σ {r.mean()-2*r.std():.2f}%")

ax.legend(fontsize=8)
ax.set_xlabel("retorno diário (%)"); ax.set_ylabel("nº de pregões")
ax.set_title("Distribuição dos retornos diários", fontweight="bold")
fig.tight_layout(); fig.savefig("hist.png", dpi=150)
```

Resultado

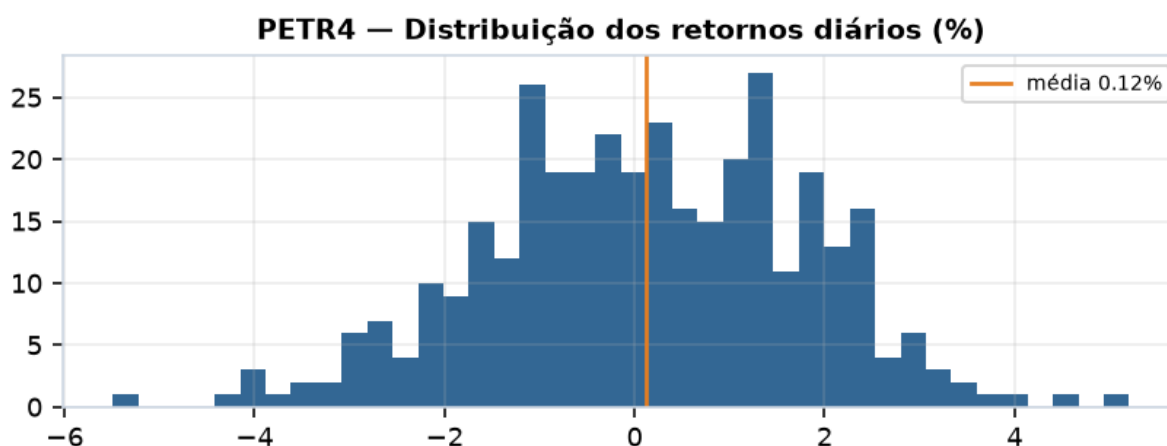


Figura 17.1 — Distribuição concentrada perto de zero, com eventos extremos nas pontas.

A largura da distribuição é a volatilidade: quanto mais espalhada, mais arriscado o ativo. A linha de -2σ é a base intuitiva do VaR (Value at Risk): em condições normais, cerca de 2,5% dos dias ficam abaixo dela.

18. Dividendos por ação

Quem paga mais proventos.

Um ranking simples, mas essencial para o investidor de renda. Some os proventos do ano por ativo e ordene.

A consulta SQL

SQL

```
SELECT d.ticker,
       ROUND(SUM(d.valor), 2)           AS total_por_acao, -- R$ por ação no ano
       COUNT(*)                        AS pagamentos,      -- quantas vezes pagou
       ROUND(SUM(d.valor) * k.quantidade, 2) AS recebido_carteira -- o que VOCÊ recebeu
FROM dividendos d
LEFT JOIN carteira k ON k.ticker = d.ticker -- LEFT: mantém ativos fora da carteira
WHERE d.data >= '2025-01-01' AND d.data <= '2025-12-31' -- janela do ano
GROUP BY d.ticker
ORDER BY total_por_acao DESC; -- maior pagador primeiro
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

fig, ax = plt.subplots(figsize=(7, 3.2))
barras = ax.bar(df["ticker"], df["total_por_acao"], color="#0f8a5f", width=0.6)

ax.bar_label(barras,
             fmt="R$ %.2f", fontsize=7, padding=2) # rótulo automático no topo das barras
                                                    # (matplotlib >= 3.4)

ax.set_ylabel("R$ por ação")
ax.set_ylim(0, df["total_por_acao"].max() * 1.18) # 18% de folga para o rótulo não encostar
ax.set_title("Dividendos pagos em 2025", fontweight="bold")
fig.tight_layout(); fig.savefig("dividendos.png", dpi=150)
```

Resultado

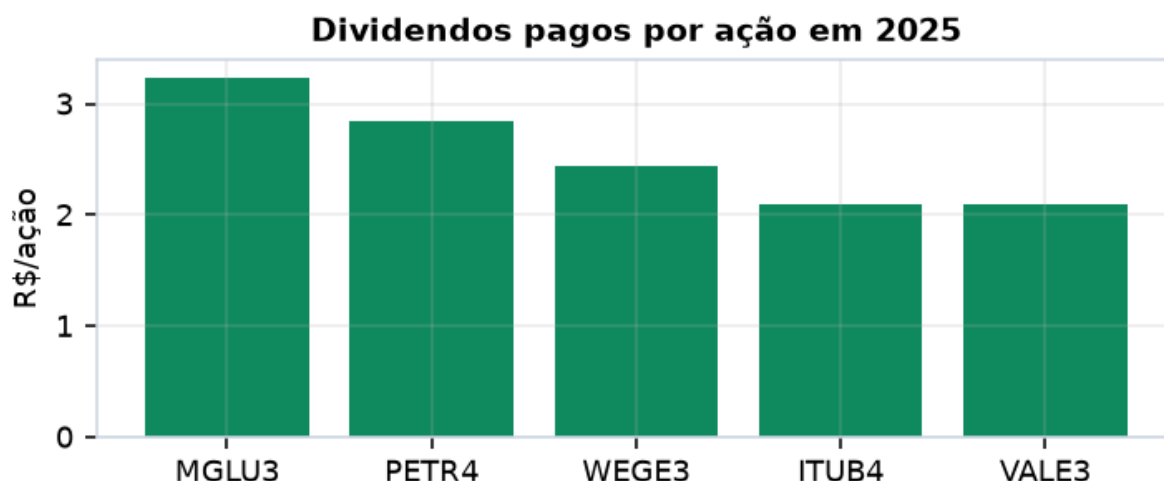


Figura 18.1 — Total de proventos por ação no ano, ativo a ativo.

`ax.bar_label()` substitui o laço manual de `ax.text()` e é o jeito moderno de rotular barras. Lembre-se de aumentar o `ylim`, senão o texto da barra mais alta é cortado.

19. Juros compostos (simulação)

Projeção de aportes mensais.

Nem todo gráfico vem do banco: simulações são igualmente úteis. Aqui projetamos o patrimônio de aportes mensais e destacamos, em área verde, a parte que é **rendimento** — e não dinheiro do seu bolso.

A consulta SQL

SQL

```
-- Dá para simular no próprio SQLite com uma CTE recursiva:
WITH RECURSIVE projecao(mes, saldo) AS (
    SELECT 1, 500.0                                -- caso base: 1º mês, primeiro aporte
    UNION ALL
    SELECT mes + 1,                                -- passo recursivo
           saldo * 1.008 + 500.0                    -- rende 0,8% e recebe novo aporte
    FROM projecao
    WHERE mes < 240                                -- para em 240 meses (20 anos)
)
SELECT mes,
       ROUND(saldo, 2) AS patrimonio,
       mes * 500      AS total_aportado,
       ROUND(saldo - mes*500, 2) AS juros_ganhos
FROM projecao;
```

O código Python comentado

PYTHON

```
import numpy as np
meses = np.arange(0, 241)                # 0 a 240 meses = 20 anos
aporte, taxa = 500, 0.008                # R$ 500/mês a 0,8% ao mês

# Fórmula do montante de uma série de aportes (valor futuro de anuidade):
patrimonio = aporte * (((1 + taxa) ** meses - 1) / taxa)
aportado = aporte * meses                  # soma simples do que saiu do bolso

fig, ax = plt.subplots(figsize=(9, 3.6))
ax.plot(meses/12, patrimonio, color="#0f4c81", lw=1.8, label="Patrimônio (0,8% a.m.)")
ax.plot(meses/12, aportado, color="#64748b", lw=1.2, ls="--", label="Total aportado")
ax.fill_between(meses/12, aportado, patrimonio, # área entre as duas curvas...
               color="#0f8a5f", alpha=0.18)    # ...é exatamente o juro acumulado

from matplotlib.ticker import FuncFormatter
ax.yaxis.set_major_formatter(FuncFormatter(lambda x, p: f"R$ {x:,.0f}".replace(",", ".")))
ax.set_xlabel("anos"); ax.legend(fontsize=8)
ax.set_title("Juros compostos – R$ 500 por mês", fontweight="bold")
fig.tight_layout(); fig.savefig("juros.png", dpi=150)
```

Resultado

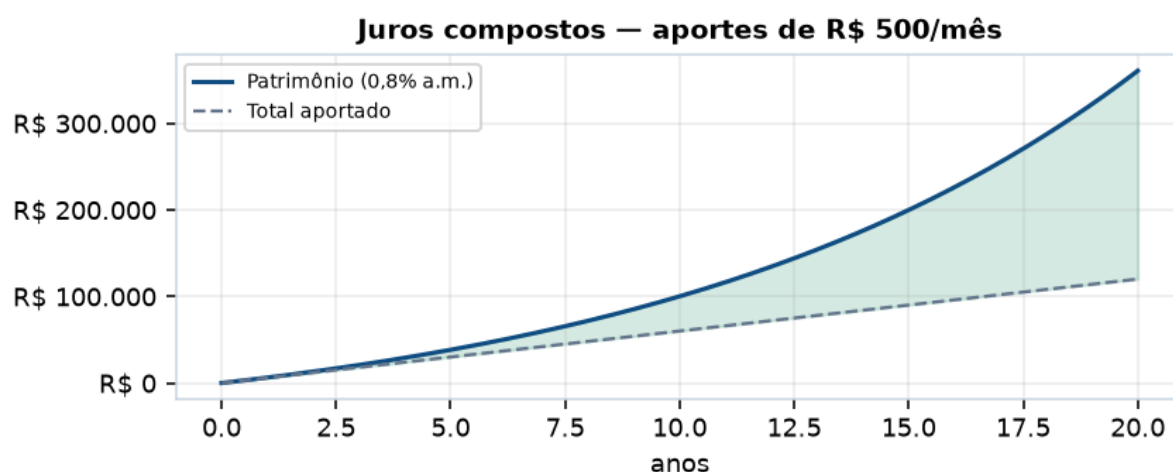


Figura 19.1 — A área verde (juros) ultrapassa o aportado por volta do 12º ano.

O momento em que a área verde fica maior que a linha tracejada é a virada do jogo: a partir dali seus rendimentos trabalham mais do que seus aportes. É o gráfico mais persuasivo de qualquer apresentação sobre investimento de longo prazo.

20. Fluxo de caixa: barras + linha acumulada

Dois eixos Y no mesmo gráfico.

Receitas e despesas em barras opostas, saldo acumulado em linha. Como as grandezas têm escalas diferentes, usamos um segundo eixo Y com `twinx()`.

A consulta SQL

SQL

```
SELECT substr(data, 1, 7) AS mes,
       ROUND(SUM(CASE WHEN valor > 0 THEN valor ELSE 0 END), 2) AS receitas, -- entradas
       ROUND(SUM(CASE WHEN valor < 0 THEN -valor ELSE 0 END), 2) AS despesas, -- saídas (positivas)
       ROUND(SUM(valor), 2) AS resultado -- saldo do mês
FROM lancamentos
GROUP BY mes
ORDER BY mes;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)
df["acumulado"] = df["resultado"].cumsum() # cumsum = soma acumulada mês a mês

fig, ax = plt.subplots(figsize=(9, 3.8))
ax.bar(df.mes, df.receitas, color="#0f8a5f", width= 0.4, align="edge", label="Receitas")
ax.bar(df.mes, -df.despesas, color="#c0392b", width=-0.4, align="edge", label="Despesas")
# width negativo + align="edge" coloca a segunda barra à ESQUERDA da posição: barras lado a lado

ax2 = ax.twinx() # cria um eixo Y independente, à direita
ax2.plot(df.mes, df.acumulado, color="#0f4c81", lw=1.8, marker="o", ms=4,
         label="Saldo acumulado")
ax2.grid(False) # evita duas grades sobrepostas

ax.legend(fontsize=7, loc="upper left") # cada eixo tem sua própria legenda
ax2.legend(fontsize=7, loc="lower right")
ax.set_title("Fluxo de caixa mensal", fontweight="bold")
fig.tight_layout(); fig.savefig("fluxo.png", dpi=150)
```

Resultado

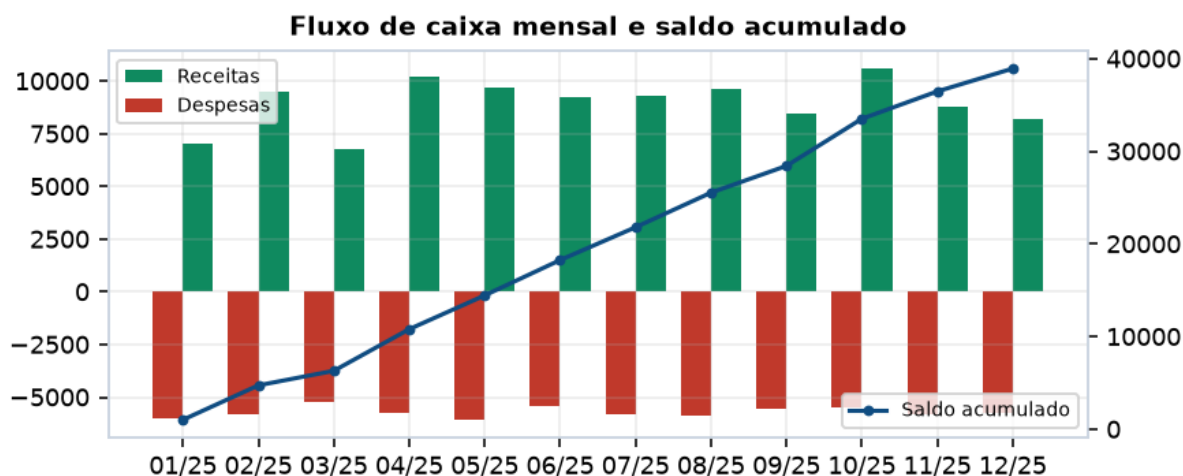


Figura 20.1 — Barras (eixo esquerdo) e saldo acumulado (eixo direito).

Dois eixos Y exigem responsabilidade: escolhendo escalas convenientes é possível 'provar' qualquer narrativa. Use apenas quando as grandezas forem realmente diferentes e deixe claro, pela cor, qual série pertence a qual eixo.

21. Boxplot de volatilidade

Comparando o risco de vários ativos.

Cada caixa resume a distribuição de retornos de um ativo: a linha central é a mediana, a caixa cobre 50% dos dias (quartis 1 a 3), as hastes vão até 1,5 vez o intervalo interquartil e os pontos soltos são outliers — os dias de pânico ou euforia.

A consulta SQL

SQL

```
SELECT ticker,
       (fechamento / LAG(fechamento) OVER (PARTITION BY ticker ORDER BY data) - 1) * 100
       AS retorno_pct
FROM cotacoes
ORDER BY ticker, data;
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con).dropna()
grupos = [g["retorno_pct"].values                # uma lista de arrays, um por ativo
          for _, g in df.groupby("ticker")]
nomes = sorted(df["ticker"].unique())

fig, ax = plt.subplots(figsize=(8, 3.4))
ax.boxplot(grupos,
           tick_labels=nomes,                    # rótulo de cada caixa (matplotlib >= 3.9)
           patch_artist=True,                   # permite pintar o interior da caixa
           boxprops=dict(facecolor="#d9e1f2", color="#0f4c81"),
           medianprops=dict(color="#c0392b", linewidth=1.4), # mediana em destaque
           flierprops=dict(marker="o", markersize=2, alpha=0.4)) # outliers discretos

ax.axhline(0, color="#94a3b8", lw=0.8)
ax.set_ylabel("retorno diário (%)")
ax.set_title("Dispersão dos retornos por ativo", fontweight="bold")
fig.tight_layout(); fig.savefig("box.png", dpi=150)
```

Resultado

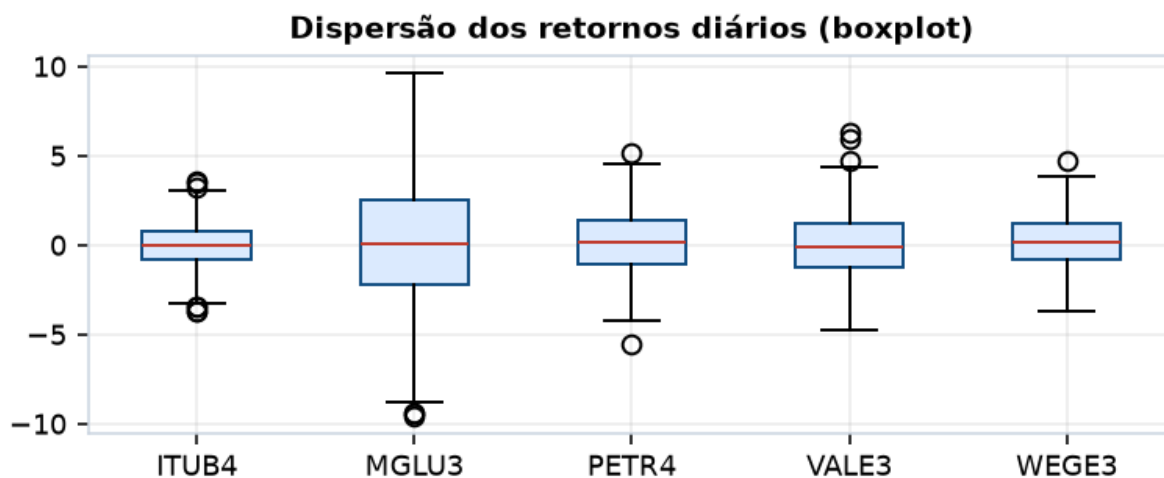


Figura 21.1 — Caixas mais altas indicam ativos mais voláteis.

Em versões do matplotlib anteriores à 3.9 o parâmetro se chama `labels` em vez de `tick_labels`. É um erro comum ao rodar o mesmo script em máquinas diferentes.

22. Sazonalidade por dia da semana

Um caso clássico de agregação por data.

O SQLite sabe extrair partes de uma data ISO com `strftime`. Isso abre a porta para análises de sazonalidade: dia da semana, mês do ano, trimestre.

A consulta SQL

SQL

```
SELECT CASE strftime('%w', data)      -- '%w' devolve 0=domingo, 1=segunda ... 6=sábado
        WHEN '1' THEN 'Seg'
        WHEN '2' THEN 'Ter'
        WHEN '3' THEN 'Qua'
        WHEN '4' THEN 'Qui'
        WHEN '5' THEN 'Sex'
      END
      ROUND(AVG(volume))              AS volume_medio,
      COUNT(*)                        AS pregoes
FROM cotacoes
WHERE ticker = 'WEGE3'
  AND strftime('%w', data) BETWEEN '1' AND '5' -- descarta fins de semana, se houver
GROUP BY strftime('%w', data)           -- agrupa pelo número (mantém a ordem correta)
ORDER BY strftime('%w', data);
```

O código Python comentado

PYTHON

```
df = pd.read_sql_query(sql, con)

fig, ax = plt.subplots(figsize=(7, 3.0))
barras = ax.bar(df["dia_semana"], df["volume_medio"], color="#e67e22", width=0.6)

media_geral = df["volume_medio"].mean()
ax.axhline(media_geral, color="#334155", ls="--", lw=1,          # referência: média dos 5 dias
            label="média geral")

from matplotlib.ticker import FuncFormatter
ax.yaxis.set_major_formatter(FuncFormatter(lambda x, p: f"{x/1e6:.0f}M")) # milhões, legível
ax.legend(fontsize=7)
ax.set_title("Volume médio por dia da semana", fontweight="bold")
fig.tight_layout(); fig.savefig("dow.png", dpi=150)
```

Resultado

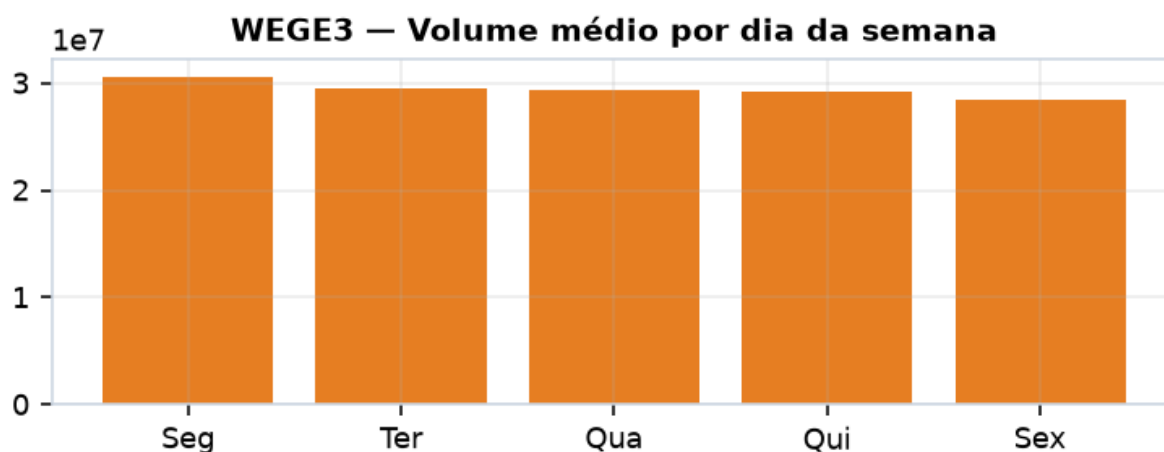


Figura 22.1 — Comparação do volume negociado em cada dia útil.

Agrupe por `strftime('%w', data)` e apenas **exiba** o nome com CASE. Se você agrupasse pelo texto 'Seg', 'Ter', a ordenação sairia alfabética (Qua, Qui, Seg...) e o gráfico ficaria sem sentido.

23. Leitura em lotes (chunks) de bases grandes

Quando a tabela não cabe na memória.

Uma base com dezenas de milhões de cotações não cabe num DataFrame. A solução é ler em pedaços, processar cada pedaço e guardar um **marcador** da última linha lida, para poder retomar exatamente de onde parou.

23.1 Paginação por chave (keyset)

SQL

```
-- Sempre mais rápido que LIMIT/OFFSET: usa o índice em vez de pular linhas uma a uma.
SELECT ticker, data, fechamento
FROM cotacoes
WHERE (ticker, data) > (:ultimo_ticker, :ultima_data)  -- avança a partir do marcador
ORDER BY ticker, data
LIMIT 50000;                                         -- tamanho do lote
```

PYTHON

```
tamanho = 50_000
marcador = ("", "")                                # começa antes de qualquer chave

while True:
    lote = pd.read_sql_query(
        """SELECT ticker, data, fechamento
           FROM cotacoes
           WHERE (ticker, data) > (?, ?)             -- comparação de tupla: SQLite suporta
           ORDER BY ticker, data
           LIMIT ?""",
        con, params=(marcador, tamanho)
    )
    if lote.empty:                                  # acabaram os dados
        break

    processa(lote)                                  # sua agregação parcial aqui

    ultima = lote.iloc[-1]                          # última linha do lote
    marcador = (ultima["ticker"], ultima["data"])    # novo ponto de retomada
```

23.2 O atalho do pandas: chunksize

PYTHON

```
soma, contagem = 0.0, 0                            # acumuladores da média incremental

for pedaco in pd.read_sql_query("SELECT fechamento FROM cotacoes",
                                con, chunksize=100_000): # devolve um GERADOR de DataFrames
    soma      += pedaco["fechamento"].sum()             # agrega só o pedaço atual
    contagem  += len(pedaco)                             # nunca guarda tudo na memória

print("média geral:", soma / contagem)
```

Dica de ouro

Para **desenhar** uma série gigante, não plote milhões de pontos: agregue por semana ou mês no SQL (GROUP BY strftime('%Y-%W', data)). O olho não distingue mais do que alguns milhares de pontos, e o PNG fica leve.

24. Boas práticas, desempenho e exportação

24.1 Desempenho no SQLite

- Crie índices para as colunas usadas em WHERE e ORDER BY: `CREATE INDEX idx_cot ON cotacoes(ticker, data);`
- Use `EXPLAIN QUERY PLAN SELECT ...` para conferir se o índice está sendo usado (procure por SEARCH, não SCAN).
- Rode `ANALYZE`; depois de grandes cargas para atualizar as estatísticas do otimizador.
- Agregue no banco. Trazer linhas cruas para o Python só para somar é o gargalo mais frequente.
- Uma transação por lote, não por linha: `executemany` + um único `commit`.

24.2 Segurança e correção

Atenção

Nunca monte SQL com f-string ou concatenação de texto: `f"WHERE ticker='{x}'"` abre a porta para SQL Injection e quebra com apóstrofes. Use sempre `?` e o argumento `params`.

- Valores monetários: prefira INTEGER em centavos quando precisar de exatidão contábil; REAL sofre arredondamento binário.
- Datas sempre em ISO AAAA-MM-DD.
- Feche a conexão (`con.close()`) ou use `with`.

24.3 Exportando os resultados

PYTHON

```
# --- imagem em alta resolução para relatório impresso ---
fig.savefig("grafico.png", dpi=300, bbox_inches="tight") # bbox_inches corta o excesso de margem

# --- vetorial (escala sem perder qualidade, ideal para PDF/slides) ---
fig.savefig("grafico.svg")
fig.savefig("grafico.pdf")

# --- vários gráficos num único PDF de várias páginas ---
from matplotlib.backends.backend_pdf import PdfPages
with PdfPages("relatorio.pdf") as pdf:
    for ticker in ["PETR4", "VALE3", "ITUB4"]:
        fig = desenha_grafico(ticker) # sua função que devolve uma figura
        pdf.savefig(fig)              # acrescenta uma página
        plt.close(fig)                # libera memória a cada iteração

# --- tabela para Excel, acompanhando os gráficos ---
df.to_excel("dados.xlsx", index=False, sheet_name="Cotações")
```

24.4 Um template reaproveitável

PYTHON

```

import sqlite3, pandas as pd, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BANCO = "financas.db"

def consulta(sql, params=()):
    """Executa um SELECT e devolve um DataFrame, sempre com a conexão fechada ao final."""
    with sqlite3.connect(BANCO) as con:
        return pd.read_sql_query(sql, con, params=params)

def estilo(ax, titulo, ylabel=""):
    """Padroniza a aparência de todos os gráficos do projeto."""
    ax.set_title(titulo, fontweight="bold")
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.25)
    ax.spines[["top", "right"]].set_visible(False)
    return ax

def grafico_preco(ticker, arquivo):
    df = consulta("SELECT data, fechamento FROM cotacoes WHERE ticker = ? ORDER BY data",
                  (ticker,))
    df["data"] = pd.to_datetime(df["data"])
    fig, ax = plt.subplots(figsize=(9, 3.5))
    ax.plot(df.data, df.fechamento, color="#0f4c81", lw=1.3)
    estilo(ax, f"{ticker} – Fechamento", "R$")
    fig.tight_layout(); fig.savefig(arquivo, dpi=150); plt.close(fig)

for t in ["PETR4", "VALE3", "ITUB4", "MGLU3", "WEGE3"]: # gera os 5 relatórios de uma vez
    grafico_preco(t, f"preco_{t}.png")

```

Com esse template e os 16 exemplos anteriores você tem um kit completo: o banco guarda, o SQL resume, o pandas ajusta e o matplotlib comunica. Troque os tickers pelos seus, ajuste as cores da sua marca e o relatório passa a se gerar sozinho.

Próximos passos

Sugestões: (1) automatize a carga diária de cotações; (2) crie uma view vw_carteira com o valor de mercado; (3) monte um painel único com `plt.subplots(2,2)` reunindo preço, retorno mensal, pizza e drawdown.